DSA0308 - Natural Language Processing

CIA - 1

Name : Jothika M
Reg No : 192324235
B.Tech (AI & DS).

①. Regular Expression :-

Problem statement :

Design a Regular Expression for a strict dd/mm/yyyy date., Indian mobile Number, e-mail address, Analyze why a mobile-number pattern without Anchoring or look-around false positive.

Given Text :

L1 : Appointment on 05/03/2025 at 10:30

L2 : Call 9876543210 or +911234567 89

L3 : Invalid mobile Number +91.

L4 : e-mail address (jothikam12@gmail.com)

L5 : Date : 5/3/25.

a). Regular Expression :

1. Date format :

$$\b(0[1-9]|[0-9]|[0-a]|[-\$/\&!*=:\\])[0-9]\{4\}\b$$

⇒ \b boundary , 0[1-9] Day -01 to 09, [0-12] month,

Here date format 05/03/2025 like (dd/mm/yyyy) at 10:30 So the Appointment for the hospital Information System stores clinical Note as plain text. and an NLP team must extract structured data Using Regular Expression. In this (L1) process 05/03/2025 at 10:30. (L2) Call 9876543210 or +91-9123456789. (L3) Indian e-mail Address Pattern that Used to Analyse the Mobile Number pattern. (L4) e-mail mobile Number false positive. (L5) Data format

b). Indian mobile Number:-

(?<![0-9])(\+91[-]?|0)?[6-9][0-9]{9}(?![0-9])

*. [0-9] Number (0-9) * No. digit Number.

c). Email - address.

[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}.

b).

Line.	Date	MN.	Email
L1	05/03/2025	None.	None
L2	None.	9876543210	None
L3	None	+91-9123456789	None.
L4	None.	None.	ravi.k@college.edu
~~L5~~.			

c) False positive Analysis...

(?<![0-9]).. (?![0-9]).

Occurs when an Invalid Input is Incorrectly Identified as Valid.

d) Date Substitution.

s/\b([0-9]{2})\/([0-9]{2}\/[0-9]{4}\b/.

\3-\2-\1/.

Appointment on 2025-03-05 at 10:30.

⇒. [0-9]{2} - Captures day.

⇒. [0-9]{2} - Captures months.

⇒. [0-9]{4} - Captures years.

⇒ \3-\2-\1 - Rearranges the Captured Values.

These Condition ensures that the Complete Number Contains 10 digits exactly.

∴. Appointment on 2025-03-05 at 10:30.

(2).

Given:-

* finite set - Automation for English Nominal
* Automation morpheme sequences.
* (i) dog + -s. (ii) mouse + -s, iii) geese.
  iv) fox + s, (v) mice + -s.
* show the lexical.

a) Design a finite set:-

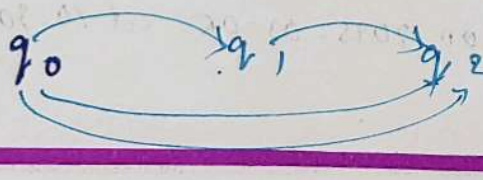
. used to Recognize string based on a set of states and transitions.

States :-

q_0 - Inital set.
q_1 - Regular singular noun state.
q_2 - plural noun state.

Transitions :-

$q_0 - q_1$ - Regular noun.
$q_0 - q_2$ - Irregular.
$q_0 - q_2$ - Irregular.
$q_1 - q_2$ - Add plural suffix - s.



b). Acceptance :-

Sequence.	Path	Result.
dog +s.	$q0 \to q1 \to q2$	Accept.
mouse +-s	$q0 - q2$ no futher	Rejected.
geese.	$q0 \to q2$	Accept.
fox +-s	$q0 \to q1 \to q2$	Accept.
mice +-s	$q0 \to q2$, no futher	Rejected.

c). Finite set Morphological:- Parsing :-

Process of Analyzing a word into its root word and grammatical Information. Using finite state transducers.

Three levels :-

1. Lexical level
2. Intermediate level.
3. Surface level.

Word	Lexical Level	Intermediate	Surface.
foxes.	fox +N + PL	fox^s#	foxes.
cats	cat +N+ PL	cat^s#.	cats
geese.	goose +N+PL	geese #	geese.

Orthographic Rule:-

$$\varepsilon \rightarrow e / \{x, s, z\} \hat{}__ s\#.$$

d). Inflectional vs Derivational Morphology.

Word	Type.	Justification
walked.	Inflectional.	-ed indicated past tense without changing word class.
Computerization	Derivational	Creates a new word through multiple derivational suffixes.

Inflectional Morphology produces grammatical variations, whereas derivational Morphology creates new words.

(3). Porter Stemmer, N-grams & Entropy :-

(i). Porter stemming Algorithm:

Rule-based technique in Natural language Processing is used to Reduce Words to their root or stem form by Removing prefixes & suffixes.

Algorithm:-

* . Read I/P.
* . Convert the Word into lowercase.
* . Identify the Suffix.
* . Apply the Porter stemming Rule.
* . Remove Unnecessary Suffix.
* . Return to final stem.

Word.	stemming word steps.	Final stem.
Ponies	Step 1a: IES → I	poni
motoring.	step 1b: ING - ε	motor.
Relational	step 2 : ATIONAL ->ATE give relate.	relat.

hopefulness	step 2: NESS - e give. hopeful : FULL give hope	hope.
generalizations	step 1: S - IZATION. IZE.	general.

ii). Word Counting, N-grams & Entropy.

Given:-

S_1 : the cat sat.

S_2 : the dog sat.

S_3 : the cat ran

a). Unigram & Bigram Count :-

↓ ↓

A single word Sequence of two Consecutive.

Unigram Count	Count
<s>	3
the.	3
cat	2
dog	1
sat	2
ran	1
</s>	3

Bigram	Count
<s> the	3
the cat	2
the dog.	1
cat sat	1
cat ran.	1
dog sat	1
sat </s>	2
ran </s>	1

b): Calculation:-

$$P(w_i / w_{i-1}) = \frac{C(w_{i-1}, w_i)}{C(w_{i-1})}$$

$$P(S) = \prod_i P(w_i / w_{i-1})$$

Sen 1 : the cat sat

$$P(S_1) = P(\text{the} / <s>) \times P(\text{cat} / \text{the}) \times P(\text{sat} / \text{cat}) \times P(</s> / \text{sat})$$

$$= 1 \times \frac{2}{3} \times \frac{1}{2} \times 1$$

$$= \frac{1}{3} = 0.333.$$

sen 2 : the dog sat

$$P(S_2) = 1 \times \frac{1}{3} \times 1 \times 1$$

$$= \frac{1}{3} = 0.333$$

sen 3 : the dog ran

$$P(S_3) = 1 \times \frac{1}{3} \times 0 \times 1$$

$$= 0.$$

Table :-

sen	Probability.	Result
S_1 : the cat sat	0.333	Non-zero
S_2 : the dog sat	0.333	Non-zero
S_3 : the dog ran	0	0.

c). Entropy & Perplexity.

Entropy. measures the Uncertainty of a language model when predicting the next word.

Formula:

$$H = -\frac{1}{N} \log_2 P(S)$$

For S_1 : SP : 1/3

No. of token : 4.

$$H = -\frac{1}{4}\log_2\left(\frac{1}{3}\right) = \frac{1}{4} \times 1.585$$

$$\boxed{H = 0.396 \text{ bits/word}}$$

Perplexity :-

$$PP = 2^{H}$$

$$PP = 2^{0.396}$$

$$\boxed{PP \approx 1.32}$$

d) Evaluation :-

* S3 receives zero probability because the bigram dog ran was not observed in the training corpus.

N-gram models calculate sentence probability based on word sequences.

---

(4). Language Model Smoothing :-

Given :-

<s> I like tea </s>

<s> I like coffee </s>

<s> I drink tea </s>.

a) Counts & Max :-

Maximum Likelihood Estimation (MLE) :-

MLE is a probability of word based sequence.

of frequency in the training Corpus.

$$P(w/v) = \frac{C(v,w)}{C(v)}$$

Unigram word	Count
<s>	3
I	3
like	2
tea	2
coffee	1
drink	1
</s>	3

Bigram word	Count
<s> I	3
I like	2
I drink	1
like tea	1
like coffee	1
drink tea	1
tea </s>	2
coffee </s>	1

Calculate P(tea / like):

$$P(\text{tea} / \text{like}) = \frac{1}{2} = 0.5$$

Calculate p(drink/like): $p(\text{drink} / \text{like}) = \frac{0}{2} = 0.$

b). Add-one (Laplace):-

Add-one smoothing adds 1 to every digram count to avoid zero probabilities.

$$P(w/v) = \frac{C(v,w)+1}{C(v)+V}$$

$$P(\text{tea}/\text{like}) = \frac{1+1}{2+6} = \frac{2}{8}$$

$$= 0.25$$

$$P(\text{drink}/\text{like}) = \frac{0+1}{2+6} = \frac{1}{8} = 0.125$$

c). Katz Backoff Smoothing:-

Katz Backoff is smoothing technique that uses lower order probabilities when a digram is not found in training corpus.

$d = 0.5$

$$P^*(w/v) = \frac{C(v,w)-d}{C(v)}$$

$$P^*(\text{tea}/\text{like}) = \frac{1-0.5}{2}$$

$$= 0.25$$

$$P^*(\text{coffee}/\text{like}) = \frac{1-0.5}{2}$$

$$= 0.25$$

$$\beta(\text{like}) = 1 - (0.25 + 0.25)$$

$$\beta(\text{like}) = 0.5$$

$P(I) = \frac{3}{12} = 0.25$

$P(\text{like}) = \frac{2}{12} = \frac{1}{6}$.

$P(\text{drink}) = \frac{1}{12}$

$P(</s>) = \frac{3}{12} = 0.25$

$1 - P(\text{tea}) - P(\text{coffee})$

$= 1 - \frac{2}{12} - \frac{1}{12} = 0.75$

$\alpha(\text{like}) = \frac{0.5}{0.75}$

$= \frac{2}{3}$

$P(\text{drink} / \text{like}) = \alpha(\text{like} \times P(\text{drink})$.

$= \frac{2}{3} \times \frac{1}{12}$ $\boxed{= 0.056.}$

$P(\text{tea} / \text{like}) = 0.25$

d). Deleted :-

$P(w/v) = \lambda_1 P_{\text{bigram}}(w/v) + \lambda_2 P_{\text{unigram}}$.

$\lambda_1 = 0.7$, $\lambda_2 = 0.3$

$= 0.7(0) + 0.3\left(\frac{1}{12}\right)$

$= 0 + 0.025 = 0.025$.

$P(t / \text{like}) = 0.7(0.5) + 0.3\left(\frac{2}{12}\right)$

$= 0.35 + 0.05$

$\boxed{= 0.60}$

Deleted. Interpolation or Katz Backoff is preferable.

5. Part of Speech (POS) Tagging

Problem statement:

Apply different part-of-speech tagging approaches to the sentence "They can fish". Calculate the most probable tag sequence using the Viterbi Algorithm:

Given:-

They can fish

Tag	Meaning.
PRP	Personal Pronoun.
MD	Model Verb.
VB	Verb Based
NN	Noun.

a). Viterbi Algorithm:-

The dynamic programming Algorithm used in Hidden Markov Models (HMMs) to find the most popular probable sequence of hidden states (POS tags)

Formula:-

$$V(t)(s) = \max_{s'} [V_{t-1}(s') \times P(s|s')] \times P(w_t|s)$$

step1:-

$V_1(PRP) = 0.5 \times 0.8$

$\boxed{V_1(PRP) = 0.4}$

step2:

Tag	Calculation	Probability
MD	$0.4 \times 0.4 \times 0.3$	0.048
VB	$0.4 \times 0.5 \times 0.02$	0.004
NN	$0.4 \times 0.1 \times 0.01$	0.0004

step3:

$$V_3(VB) = \max \begin{cases} 0.048 \times 0.8 \\ 0.004 \times 0.1 \times 0.01 \\ 0.0004 \times 0.4 \end{cases}$$

$$= \max(0.0384, 0.0004, 0.00016) \times 0.01$$

$$\boxed{V_3(VB) = 0.000384}$$

$$V_3(NN) = \max \begin{cases} 0.048 \times 0.2 \\ 0.004 \times 0.5 \times 0.02 \\ 0.0004 \times 0.3 \end{cases}$$

$$\boxed{V_3(NN) = 0.000192}$$

Step 4:-

Final tag	Prob.
VB	0.000384
NN.	0.000192

$$P = 3.84 \times 10^{-4}.$$

Rule-based Pos Tagging.

⇒ They can fish.

⇒ They / PRP can / MD.

⇒. fish / VB.

Output: They / PRP can / MD fish / VB

Score = Fixes - Breaks.

Fixes = 46.

Breaks = 6.

Score = 46 - 6.

Score = 40.

Pos tagging Identified the grammer role. of each word in sentences.